



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



**TFG del Grado en Ingeniería
Informática**

**Análisis comparativo de
arquitecturas de redes
neuronales convolucionales
para la detección de pose, la
clasificación y evaluación de
enfermedades
neurodegenerativas, como la
esclerosis lateral amiotrófica,
en imágenes 2D.
Documentación Técnica**



Presentado por Samuel Murillo Arce
en Universidad de Burgos — 7 de julio de 2026
Tutor: Fernando Rivas Navazo

Índice general

Índice general	i
Índice de figuras	iii
Índice de tablas	iv
Apéndice A Plan de Proyecto Software	1
A.1. Introducción	1
A.2. Planificación temporal	1
A.3. Estudio de viabilidad	4
Apéndice B Especificación de Requisitos	7
B.1. Introducción	7
B.2. Objetivos generales	7
B.3. Catálogo de requisitos	8
Apéndice C Especificación de diseño	11
C.1. Introducción	11
C.2. Diseño de datos	11
C.3. Diseño arquitectónico	13
C.4. Diseño procedimental	14
Apéndice D Documentación técnica de programación	17
D.1. Introducción	17
D.2. Estructura de directorios	17
D.3. Requisitos de desarrollo	18
D.4. Instalación del entorno	19

D.5. Punto de entrada de la aplicación	20
D.6. Interfaz gráfica	20
D.7. Gestión de vídeo	21
D.8. Modelo de datos	22
D.9. Cálculo de métricas	22
D.10.Detección de anomalías	23
D.11.Exportación de resultados	24
D.12.Mantenimiento y ampliación	24
D.13.Pruebas del sistema	24
Apéndice E Documentación de usuario	25
E.1. Introducción	25
E.2. Requisitos de usuario	25
E.3. Instalación	26
E.4. Manual del usuario	27
Apéndice F Anexo de sostenibilización curricular	31
F.1. Introducción	31
Bibliografía	33

Índice de figuras

C.1. Diseño arquitectónico de la aplicación.	13
C.2. Flujo general de procesamiento de un vídeo.	15

Índice de tablas

A.1. Estimación económica del proyecto.	5
A.2. Licencias de las tecnologías utilizadas.	6
C.1. Principales entidades manejadas por la aplicación.	12

Apéndice A

Plan de Proyecto Software

A.1. Introducción

El presente apéndice describe la planificación seguida durante el desarrollo del Trabajo Fin de Grado, cuyo objetivo principal consiste en el diseño e implementación de una aplicación de escritorio orientada al análisis comparativo de diferentes modelos de estimación de pose humana aplicados al estudio del movimiento en pacientes con enfermedades neurodegenerativas, especialmente la Esclerosis Lateral Amiotrófica (ELA).

Durante el desarrollo se adoptó una metodología de trabajo iterativa basada en principios ágiles. El proyecto se dividió en diferentes Sprints que permitieron organizar el trabajo en pequeñas tareas, facilitando la planificación, la validación continua y la adaptación a los cambios surgidos durante el desarrollo.

Cada Sprint se orientó a alcanzar objetivos concretos relacionados tanto con la implementación de la aplicación como con la investigación bibliográfica y la elaboración de la memoria del proyecto.

A.2. Planificación temporal

El proyecto se desarrolló durante aproximadamente seis meses siguiendo una planificación incremental. Aunque se estableció una planificación inicial, ésta fue adaptándose conforme avanzó el desarrollo, especialmente durante la fase de integración de los diferentes modelos de estimación de pose y la validación de los resultados obtenidos.

La organización mediante Sprints permitió mantener un seguimiento continuo del progreso del proyecto y priorizar aquellas funcionalidades consideradas fundamentales para alcanzar los objetivos planteados.

Sprint 1 (03/02/2026 – 16/02/2026)

- Definición del alcance del Trabajo Fin de Grado.
- Estudio inicial sobre enfermedades neurodegenerativas, con especial atención a la Esclerosis Lateral Amiotrófica (ELA).
- Investigación preliminar sobre técnicas de Visión por Computador aplicadas al análisis del movimiento humano.
- Búsqueda bibliográfica de artículos científicos relacionados con redes neuronales convolucionales y estimación de pose.
- Elaboración de la planificación inicial del proyecto.
- Redacción del borrador de los apartados de Introducción y Objetivos.

Sprint 2 (17/02/2026 – 02/03/2026)

- Estudio detallado del funcionamiento de las redes neuronales convolucionales.
- Investigación sobre arquitecturas modernas de estimación de pose humana.
- Comparación preliminar entre MediaPipe Pose, MoveNet Lightning y YOLOv8-Pose.
- Análisis de diferentes formatos de representación de keypoints.
- Selección inicial de las tecnologías que formarían parte del proyecto.
- Redacción del capítulo correspondiente a los conceptos teóricos.

Sprint 3 (03/03/2026 – 16/03/2026)

- Diseño de la arquitectura general de la aplicación.
- Definición de los módulos principales del sistema.

- Selección del lenguaje Python como plataforma de desarrollo.
- Evaluación de bibliotecas gráficas disponibles para aplicaciones de escritorio.
- Selección de PySide6 como framework para la interfaz gráfica.
- Inicio del desarrollo de la estructura del proyecto.

Sprint 4 (17/03/2026 – 30/03/2026)

- Desarrollo de la interfaz gráfica principal.
- Implementación del sistema de carga de vídeos.
- Desarrollo del gestor de reproducción de vídeos.
- Diseño del flujo general de procesamiento frame a frame.
- Definición de la estructura utilizada para almacenar los keypoints detectados.

Sprint 5 (31/03/2026 – 13/04/2026)

- Implementación de la arquitectura basada en detectores independientes.
- Desarrollo de la clase base para los distintos modelos de estimación de pose.
- Integración del primer detector de pose.
- Visualización del esqueleto sobre los fotogramas procesados.
- Primeras pruebas funcionales de la aplicación.

Sprint 6 (14/04/2026 – 27/04/2026)

- Desarrollo del módulo encargado del cálculo de métricas biomecánicas.
- Implementación del cálculo de ángulos articulares.
- Cálculo del rango de movimiento.
- Implementación de la velocidad de desplazamiento de las articulaciones.
- Diseño del sistema de resumen estadístico de resultados.

Sprint 7 (28/04/2026 – 11/05/2026)

- Desarrollo del sistema de detección de anomalías sobre los keypoints.
- Implementación de la exportación de resultados en formato CSV.
- Diseño del sistema de exportación en formato JSON.
- Optimización del procesamiento de vídeos.
- Validación del correcto funcionamiento de la interfaz gráfica.

Sprint 8 (12/05/2026 – 06/07/2026)

- Pruebas finales utilizando diferentes vídeos de rehabilitación.
- Comparación del comportamiento de los distintos modelos de estimación de pose.
- Corrección de errores detectados durante la fase de validación.
- Finalización de la memoria del Trabajo Fin de Grado.
- Elaboración de anexos, revisión del proyecto y preparación de la entrega.

A.3. Estudio de viabilidad

El estudio de viabilidad permite determinar si el proyecto puede desarrollarse de forma satisfactoria desde el punto de vista económico, técnico y legal. En este caso se analiza el coste estimado que supondría desarrollar la aplicación en un entorno profesional, así como las implicaciones derivadas del uso de las diferentes herramientas software empleadas durante el desarrollo.

Viabilidad económica

Aunque el presente Trabajo Fin de Grado ha sido desarrollado con fines académicos y sin ánimo de lucro, resulta interesante estimar el coste que supondría desarrollar una aplicación de características similares en un entorno profesional.

Para ello se consideran tanto los recursos humanos como el hardware y el software utilizados durante el desarrollo.

Concepto	Descripción	Coste (€)
Ingeniero de Software	Desarrollo durante 6 meses	18000
Equipo informático	Ordenador portátil de altas prestaciones	1800
Monitor externo	Monitor de 27 pulgadas	250
Licencia Windows 11 Pro	Sistema operativo	180
Electricidad e Internet	Consumo aproximado durante el desarrollo	350
Material de oficina	Documentación y otros recursos	120
Coste total estimado		20700

Tabla A.1: Estimación económica del proyecto.

La mayor parte del coste corresponde al desarrollo software, ya que representa el recurso más importante dentro del proyecto. El resto de costes corresponden principalmente al equipamiento necesario para ejecutar modelos de visión por computador y realizar las pruebas con los distintos vídeos utilizados durante la evaluación.

A pesar del coste estimado, se considera que el proyecto resulta económicamente viable debido al potencial interés que presenta como herramienta de apoyo al análisis del movimiento en pacientes con enfermedades neurodegenerativas y como plataforma para futuras investigaciones.

Viabilidad legal

Durante el desarrollo del proyecto únicamente se han empleado herramientas y bibliotecas distribuidas bajo licencias de software libre o de código abierto, permitiendo su utilización con fines docentes, de investigación y desarrollo.

La siguiente tabla resume las principales tecnologías utilizadas junto con el tipo de licencia asociado.

Herramienta	Licencia	Uso
Python	PSF License	Desarrollo de la aplicación
PySide6	LGPL v3	Interfaz gráfica
OpenCV	Apache 2.0	Procesamiento de vídeo
NumPy	BSD 3-Clause	Cálculo numérico
Pandas	BSD 3-Clause	Tratamiento de datos
Matplotlib	PSF License	Representación gráfica
Ultralytics (YOLOv8)	AGPL v3	Estimación de pose
MoveNet (TensorFlow Lite)	Apache 2.0	Estimación de pose
TensorFlow Lite	Apache 2.0	Inferencia de modelos

Tabla A.2: Licencias de las tecnologías utilizadas.

Todas las bibliotecas empleadas permiten su utilización con fines académicos y de investigación. No obstante, en un entorno comercial sería necesario revisar especialmente las condiciones de la licencia AGPL v3 asociada a Ultralytics, ya que establece determinadas obligaciones en caso de distribución del software.

En relación con la protección de datos, la aplicación no almacena información personal de pacientes ni realiza ningún tratamiento automatizado de datos identificativos. Los vídeos empleados durante las pruebas han sido utilizados exclusivamente con fines docentes y de investigación, respetando en todo momento la normativa vigente en materia de protección de datos personales (Reglamento General de Protección de Datos, RGPD).

Apéndice B

Especificación de Requisitos

B.1. Introducción

En este apéndice se describen los requisitos funcionales y no funcionales que debe satisfacer la aplicación desarrollada durante el Trabajo Fin de Grado.

El objetivo principal de esta especificación es definir de forma clara las funcionalidades que ofrece el sistema, así como las restricciones y condiciones necesarias para su correcto funcionamiento. Para ello se establece un catálogo de requisitos y una especificación mediante casos de uso que describen la interacción entre el usuario y la aplicación.

Los requisitos recogidos en este documento han servido como guía durante todo el proceso de desarrollo, permitiendo verificar posteriormente que la aplicación implementada cumple con los objetivos inicialmente establecidos.

B.2. Objetivos generales

El objetivo principal del proyecto consiste en desarrollar una aplicación de escritorio capaz de analizar vídeos mediante diferentes modelos de estimación de pose humana con el fin de facilitar el estudio del movimiento de pacientes con enfermedades neurodegenerativas, especialmente la Esclerosis Lateral Amiotrófica (ELA).

Para alcanzar dicho objetivo se establecen los siguientes objetivos específicos:

- Desarrollar una interfaz gráfica sencilla e intuitiva.
- Permitir la carga de vídeos desde el sistema de archivos.
- Integrar distintos modelos de estimación de pose humana.
- Procesar vídeos fotograma a fotograma.
- Representar gráficamente el esqueleto detectado sobre el vídeo.
- Obtener las coordenadas de los puntos articulares detectados.
- Calcular métricas biomecánicas relevantes para el análisis del movimiento.
- Detectar posibles anomalías o inconsistencias durante la detección de pose.
- Exportar los resultados obtenidos en formatos CSV y JSON.
- Facilitar la comparación entre distintos modelos de estimación de pose.

B.3. Catálogo de requisitos

Requisitos funcionales

- **RF-1.** El sistema deberá permitir cargar un vídeo almacenado en el equipo.
- **RF-2.** El sistema deberá permitir seleccionar el modelo de estimación de pose que se utilizará durante el procesamiento.
- **RF-3.** El sistema deberá visualizar el primer fotograma del vídeo cargado.
- **RF-4.** El sistema deberá procesar el vídeo de forma secuencial fotograma a fotograma.
- **RF-5.** El sistema deberá detectar la pose humana presente en cada fotograma.
- **RF-6.** El sistema deberá representar el esqueleto detectado sobre la imagen original.

- **RF-7.** El sistema deberá almacenar las coordenadas de todos los puntos articulares detectados.
- **RF-8.** El sistema deberá calcular métricas biomecánicas del movimiento.
- **RF-9.** El sistema deberá detectar anomalías durante el procesamiento.
- **RF-10.** El sistema deberá mostrar estadísticas generales del análisis realizado.
- **RF-11.** El sistema deberá permitir exportar los keypoints detectados en formato CSV.
- **RF-12.** El sistema deberá permitir exportar las anomalías detectadas.
- **RF-13.** El sistema deberá reiniciar el procesamiento del vídeo cuando el usuario lo solicite.
- **RF-14.** El sistema deberá permitir finalizar la aplicación liberando correctamente los recursos utilizados.

Requisitos no funcionales

- **RNF-1.** La aplicación deberá ejecutarse sobre sistemas operativos Windows.
- **RNF-2.** La interfaz gráfica deberá resultar intuitiva para usuarios sin conocimientos técnicos avanzados.
- **RNF-3.** El sistema deberá mantener una estructura modular que facilite la incorporación de nuevos modelos de estimación de pose.
- **RNF-4.** El código deberá desarrollarse siguiendo criterios de mantenibilidad y reutilización.
- **RNF-5.** El sistema deberá gestionar adecuadamente los errores producidos durante la carga y procesamiento de vídeos.
- **RNF-6.** La aplicación deberá minimizar el consumo de memoria durante el procesamiento.
- **RNF-7.** Los resultados obtenidos deberán almacenarse utilizando formatos estándar fácilmente reutilizables.

- **RNF-8.** La arquitectura deberá permitir incorporar nuevos algoritmos sin modificar el resto de módulos de la aplicación.
- **RNF-9.** El procesamiento deberá realizarse de forma suficientemente eficiente para permitir el análisis de vídeos de duración media.
- **RNF-10.** La aplicación deberá ser fácilmente portable entre diferentes equipos con Python instalado.

Apéndice C

Especificación de diseño

C.1. Introducción

En este apéndice se describe el diseño software desarrollado para la implementación de la aplicación. Se presenta la organización general del sistema, la estructura de los datos utilizados durante el procesamiento y la arquitectura software empleada para facilitar la modularidad y el mantenimiento del código.

El diseño se ha realizado siguiendo una arquitectura modular basada en la separación de responsabilidades, permitiendo que cada componente del sistema desempeñe una función concreta e independiente del resto. Esta organización facilita la incorporación de nuevos modelos de estimación de pose, la reutilización de código y la escalabilidad de la aplicación.

Asimismo, se describen los principales procedimientos que intervienen durante el procesamiento de un vídeo, desde su carga hasta la exportación de los resultados obtenidos.

C.2. Diseño de datos

La aplicación trabaja principalmente con información obtenida a partir de los fotogramas de un vídeo y de los puntos articulares detectados por los modelos de estimación de pose.

Los principales elementos de información utilizados durante la ejecución son los siguientes:

- Vídeo cargado por el usuario.
- Fotograma actual del vídeo.
- Modelo de estimación de pose seleccionado.
- Coordenadas de los keypoints detectados.
- Resultados asociados a cada fotograma.
- Métricas biomecánicas calculadas.
- Anomalías detectadas durante el procesamiento.
- Resultados exportados en formato CSV o JSON.

Cada fotograma procesado genera un conjunto de keypoints que representan la posición de las principales articulaciones del cuerpo humano. Estos datos constituyen la base para el cálculo posterior de métricas y para la detección de posibles anomalías durante el movimiento.

La información generada se mantiene en memoria durante el procesamiento y posteriormente puede almacenarse mediante los módulos de exportación implementados en la aplicación.

Entidad	Descripción	Formato
Vídeo	Archivo multimedia de entrada	MP4, AVI, MOV, MKV
Frame	Imagen individual del vídeo	OpenCV (NumPy)
Keypoint	Coordenada articular	(x, y, z)
PoseFrameResult	Resultado de un fotograma	Objeto Python
Métricas	Estadísticas del movimiento	Diccionario
Anomalías	Incidencias detectadas	Lista
Resultados	Exportación final	CSV / JSON

Tabla C.1: Principales entidades manejadas por la aplicación.

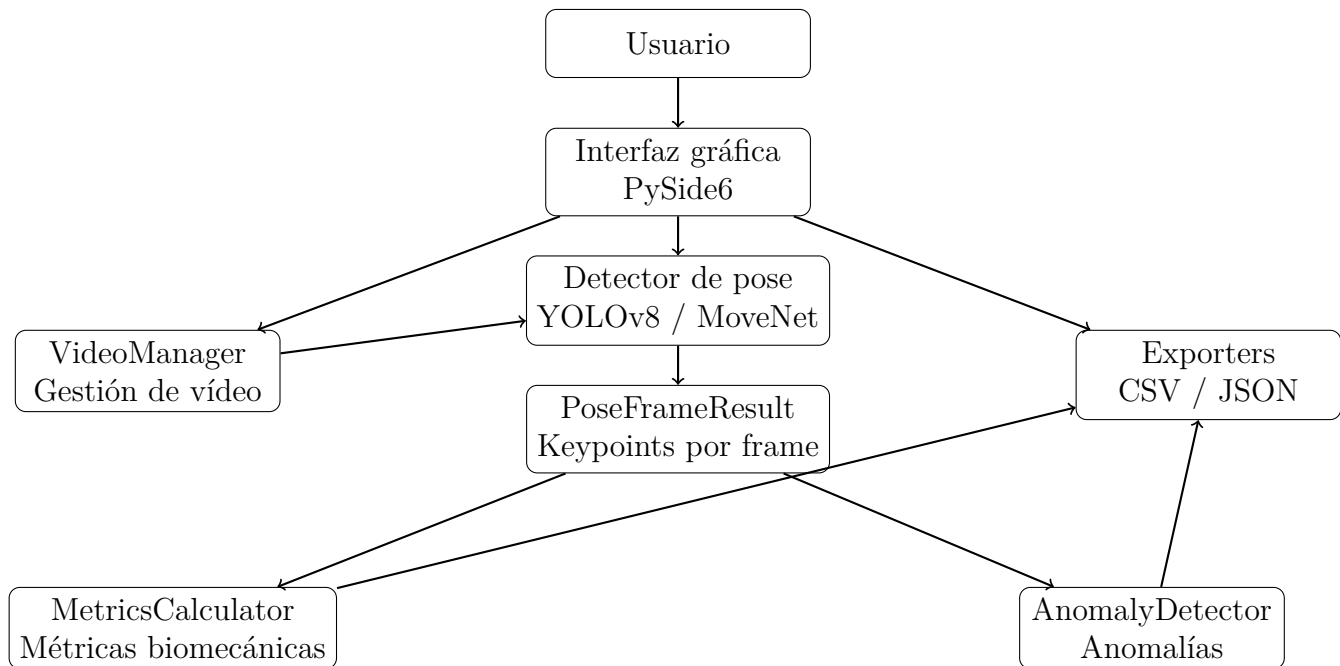


Figura C.1: Diseño arquitectónico de la aplicación.

C.3. Diseño arquitectónico

La aplicación se ha desarrollado siguiendo una arquitectura modular dividida en diferentes capas funcionales.

Cada módulo es responsable de una tarea específica, reduciendo el acoplamiento entre componentes y facilitando futuras ampliaciones del sistema.

La estructura general del proyecto se organiza en los siguientes módulos:

- **Interfaz gráfica (GUI).** Implementada mediante PySide6, permite la interacción del usuario con la aplicación.^[4]
- **Gestión de vídeo.** Encargada de cargar, reproducir y obtener los fotogramas del vídeo mediante OpenCV.^[?]
- **Modelos de estimación de pose.** Responsable de ejecutar los diferentes algoritmos de detección de pose.^[?]
- **Procesamiento de resultados.** Calcula métricas biomecánicas y detecta anomalías.^[1]

- **Exportación.** Permite almacenar los resultados obtenidos en diferentes formatos.[3]

La comunicación entre módulos se realiza mediante clases independientes, permitiendo sustituir o ampliar cualquiera de ellas sin afectar significativamente al resto de la aplicación.[2]

C.4. Diseño procedimental

El funcionamiento de la aplicación sigue una secuencia claramente definida desde la carga del vídeo hasta la exportación de los resultados.

El procedimiento general puede resumirse mediante los siguientes pasos:

1. El usuario carga un vídeo desde el sistema de archivos.
2. Se inicializa el gestor de vídeo y se obtiene el primer fotograma.
3. El usuario selecciona el modelo de estimación de pose.
4. Se inicia el procesamiento fotograma a fotograma.
5. El detector obtiene los keypoints correspondientes a cada imagen.
6. Se dibuja el esqueleto sobre el fotograma.
7. Se calculan las métricas biomecánicas.
8. Se detectan posibles anomalías en los keypoints.
9. Se actualiza la interfaz gráfica con los resultados obtenidos.
10. Finalizado el procesamiento, el usuario puede exportar los resultados en formato CSV o JSON.

Este procedimiento se repite automáticamente para todos los fotogramas del vídeo hasta finalizar el procesamiento completo.

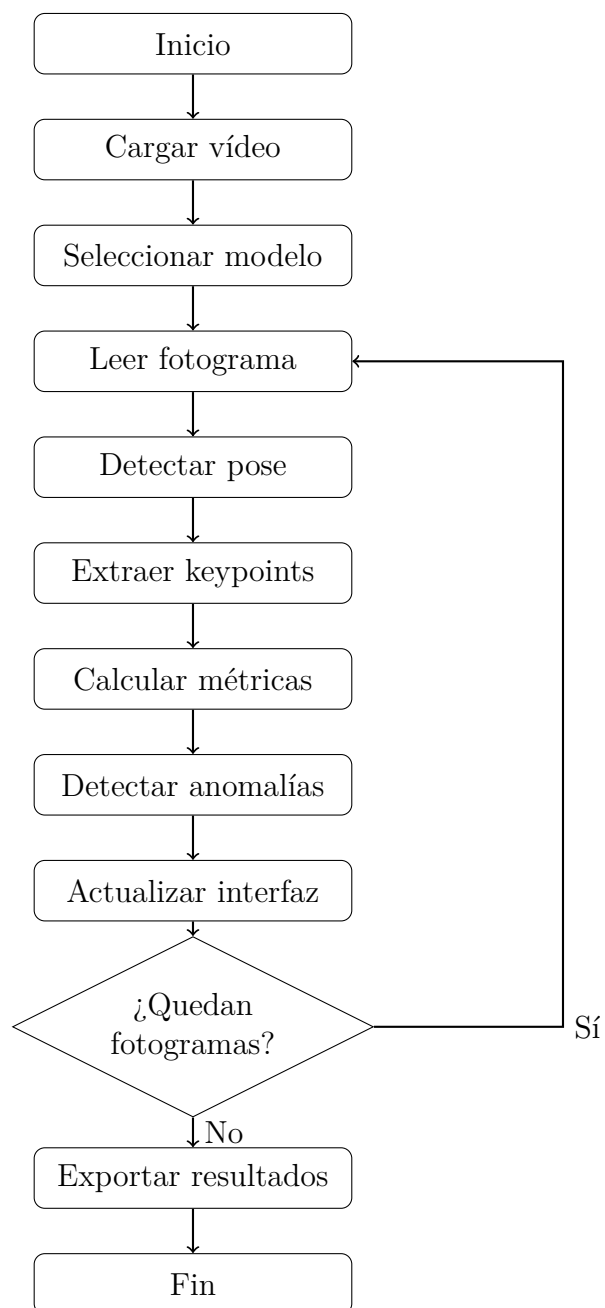


Figura C.2: Flujo general de procesamiento de un vídeo.

Apéndice D

Documentación técnica de programación

D.1. Introducción

El presente apéndice describe la estructura interna de la aplicación desarrollada, así como la organización del código fuente, las dependencias necesarias y las pautas básicas para su instalación, ejecución y ampliación.

El objetivo de este manual es facilitar que un desarrollador pueda comprender el funcionamiento interno del sistema, modificar sus componentes o incorporar nuevos modelos de estimación de pose sin necesidad de rediseñar la aplicación completa.

La aplicación ha sido desarrollada en Python y utiliza una arquitectura modular, separando la interfaz gráfica, la gestión de vídeo, los modelos de detección de pose, el cálculo de métricas y la exportación de resultados. [?]

D.2. Estructura de directorios

La estructura principal del proyecto es la siguiente:

```
PoseLab/  
|-- main.py  
|-- requirements.txt  
|-- README.md  
|-- app/
```

```
| |-- widgets/  
| | |-- video_label.py  
| |-- styles/  
| | |-- style.qss  
|-- core/  
| |-- video_manager.py  
| |-- keypoint_schema.py  
| |-- metrics.py  
| |-- anomaly_detector.py  
| |-- exporters.py  
|-- models/  
| |-- base_detector.py  
| |-- yolov8_pose_detector.py  
| |-- movenet_detector.py  
|-- data/  
| |-- input_videos/  
| |-- results/  
|-- assets/  
    |-- icons/
```

Cada carpeta agrupa componentes con una responsabilidad concreta:

- **app**: contiene la interfaz gráfica de usuario.
- **core**: contiene la lógica principal de procesamiento, métricas, anomalías y exportación.
- **models**: contiene los detectores de pose integrados en la aplicación.
- **data**: almacena vídeos de entrada y resultados generados.
- **assets**: contiene recursos gráficos auxiliares.

D.3. Requisitos de desarrollo

Para desarrollar o ejecutar la aplicación es necesario disponer de:

- Python 3.10 o superior.
- Sistema operativo Windows.

- Entorno virtual de Python.
- Bibliotecas indicadas en el archivo `requirements.txt`.
- Editor de código, preferiblemente Visual Studio Code.

Un ejemplo de archivo `requirements.txt` sería:

```
opencv-python==4.10.0.84
numpy==1.26.4
pandas==2.2.2
PySide6==6.7.2
matplotlib==3.9.1
ultralalytics
torch
torchvision
```

D.4. Instalación del entorno

Para preparar el entorno de desarrollo se recomienda crear un entorno virtual dentro de la carpeta del proyecto:

```
python -m venv .venv
```

En Windows, el entorno se activa mediante:

```
.venv\Scripts\activate
```

A continuación, se instalan las dependencias:

```
pip install -r requirements.txt
```

Finalmente, la aplicación puede ejecutarse mediante:

```
python main.py
```

D.5. Punto de entrada de la aplicación

El archivo principal del sistema es `main.py`. Este archivo se encarga de crear la aplicación Qt, cargar la hoja de estilos y mostrar la ventana principal.

```
def main() -> None:
    app = QApplication(sys.argv)
    app.setApplicationName("PoseLab")

    load_stylesheet(app)

    window = MainWindow()
    window.show()

    sys.exit(app.exec())
```

Desde este punto se inicializa la interfaz principal definida en la clase `MainWindow`.

D.6. Interfaz gráfica

La interfaz gráfica se encuentra implementada en el archivo:

`app/main_window.py`

La clase principal es `MainWindow`, que hereda de `QMainWindow`. Esta clase se encarga de:

- Crear los botones y paneles de la aplicación.
- Gestionar la carga de vídeos.
- Inicializar el modelo de estimación de pose seleccionado.
- Procesar el vídeo fotograma a fotograma.
- Actualizar la visualización del vídeo.
- Mostrar métricas y anomalías.

- Gestionar la exportación de resultados.

El procesamiento de vídeo se realiza mediante un temporizador de Qt:

```
self.timer = QTimer()
self.timer.timeout.connect(self.process_next_frame)
```

Cada vez que el temporizador se activa, se llama al método `process_next_frame`, que lee un nuevo fotograma y ejecuta el modelo de estimación de pose.

D.7. Gestión de vídeo

La gestión del vídeo se encuentra encapsulada en la clase `VideoManager`, ubicada en:

```
core/video_manager.py
```

Esta clase utiliza OpenCV para abrir el vídeo, leer fotogramas y consultar propiedades como el número total de frames, la resolución o los FPS.

Sus métodos principales son:

- `open(video_path)`: abre el archivo de vídeo seleccionado.
- `read()`: devuelve el siguiente fotograma.
- `reset()`: reinicia la lectura del vídeo.
- `current_frame_index()`: devuelve el índice del fotograma actual.
- `timestamp_for_frame(frame_id)`: calcula el instante temporal asociado a un fotograma.
- `release()`: libera los recursos del vídeo.

D.8. Modelo de datos

Los datos principales del sistema se definen en:

`core/keypoint_schema.py`

En este módulo se definen las estructuras de datos utilizadas durante todo el procesamiento de los vídeos. Estas estructuras permiten almacenar de forma organizada la información generada por los diferentes modelos de estimación de pose y facilitan posteriormente el cálculo de métricas, la detección de anomalías y la exportación de resultados.

La aplicación trabaja principalmente con dos entidades:

- **Keypoint:** representa un punto anatómico detectado sobre una persona en un fotograma determinado. Cada keypoint almacena información como el identificador del fotograma, el instante temporal, el modelo utilizado para la detección, el nombre de la articulación, sus coordenadas espaciales y el nivel de confianza asociado a la detección.
- **PoseFrameResult:** representa el resultado completo obtenido tras procesar un fotograma. Esta estructura contiene el identificador del fotograma, el instante temporal, el modelo empleado, la lista de keypoints detectados, un indicador de si se ha detectado correctamente una pose humana y el tiempo empleado durante la inferencia.

Estas estructuras permiten desacoplar la lógica de procesamiento de los modelos de estimación de pose, facilitando la incorporación de nuevos detectores sin necesidad de modificar el resto de módulos de la aplicación.

D.9. Cálculo de métricas

El cálculo de las métricas biomecánicas se realiza mediante la clase `MetricsCalculator`, ubicada en:

`core/metrics.py`

Este módulo procesa la información obtenida en cada fotograma y genera estadísticas globales sobre el movimiento analizado.

Entre las principales métricas calculadas se encuentran:

- Número total de fotogramas procesados.
- Número de fotogramas en los que se ha detectado correctamente la pose.
- Confianza media de los keypoints detectados.
- Tiempo medio de procesamiento por fotograma.
- Estimación del rendimiento del sistema en imágenes por segundo (FPS).

Estas métricas permiten comparar el comportamiento de los distintos modelos de estimación de pose y evaluar su rendimiento durante el análisis de un vídeo.

D.10. Detección de anomalías

La detección de posibles errores durante el procesamiento se implementa mediante la clase `AnomalyDetector`, localizada en:

`core/anomaly_detector.py`

Su función consiste en identificar situaciones anómalas relacionadas con la posición o la confianza de los keypoints detectados.

Entre las comprobaciones realizadas se incluyen:

- Keypoints con niveles de confianza inferiores al umbral establecido.
- Coordenadas fuera del rango esperado.
- Ausencia de determinados puntos anatómicos.
- Inconsistencias entre fotogramas consecutivos.

Las anomalías detectadas se almacenan para su posterior visualización en la interfaz gráfica y pueden exportarse junto con el resto de resultados.

D.11. Exportación de resultados

La generación de los archivos de salida se realiza mediante el módulo:

`core/exporters.py`

Actualmente la aplicación permite exportar:

- Coordenadas de los keypoints en formato CSV.
- Registro de anomalías detectadas durante el procesamiento.

La utilización de formatos estándar facilita el análisis posterior mediante herramientas estadísticas, hojas de cálculo o lenguajes de programación como Python o R.

D.12. Mantenimiento y ampliación

La arquitectura modular empleada durante el desarrollo facilita el mantenimiento del sistema y la incorporación de nuevas funcionalidades.

Para añadir un nuevo modelo de estimación de pose únicamente es necesario implementar una nueva clase que herede de `BasePoseDetector` y definir los métodos necesarios para realizar la detección y la representación gráfica de los keypoints.

Esta aproximación reduce el acoplamiento entre módulos y permite extender la aplicación sin modificar el funcionamiento del resto del sistema, favoreciendo así su escalabilidad y reutilización en futuros trabajos.

D.13. Pruebas del sistema

Las pruebas de sistema son un tipo de prueba de software en la que se evalúa el sistema completo como un todo, para asegurarse de que cumple con los requisitos funcionales y no funcionales especificados.

Las primeras pruebas consistían en la comprobación de la instalación de las aplicaciones. Después de hicieron sobre la aplicación desarrollada, en los modelos que mi ordenador permite ejecutar e instalar.

Apéndice E

Documentación de usuario

E.1. Introducción

El presente documento describe el funcionamiento de la aplicación desarrollada durante el Trabajo Fin de Grado, proporcionando al usuario toda la información necesaria para su instalación y utilización.

La aplicación ha sido diseñada para facilitar el análisis del movimiento humano mediante modelos de estimación de pose aplicados sobre vídeos. Su objetivo principal es permitir la visualización del esqueleto detectado, el cálculo de métricas biomecánicas y la exportación de los resultados obtenidos, proporcionando una herramienta de apoyo al estudio de enfermedades neurodegenerativas como la Esclerosis Lateral Amiotrófica (ELA).

El diseño de la interfaz se ha realizado buscando la sencillez y facilidad de uso, permitiendo que cualquier usuario pueda utilizar la aplicación sin necesidad de conocimientos avanzados en programación o visión por computador.

E.2. Requisitos de usuario

Para ejecutar correctamente la aplicación es recomendable disponer del siguiente entorno:

- Sistema operativo Windows 10 o Windows 11.
- Python 3.10 o superior.
- Procesador Intel Core i5 o equivalente.

- Memoria RAM mínima de 8 GB (16 GB recomendados).
- Al menos 2 GB de espacio libre en disco.
- Resolución mínima recomendada de 1280x720 píxeles.
- Ratón y teclado.

Asimismo, será necesario disponer de los paquetes software incluidos en el archivo `requirements.txt` del proyecto.

E.3. Instalación

La instalación de la aplicación consta de los siguientes pasos:

1. Descargar o clonar el repositorio del proyecto.
2. Abrir una terminal en la carpeta raíz del proyecto.
3. Crear un entorno virtual de Python:

```
python -m venv .venv
```

4. Activar el entorno virtual:

```
.venv\Scripts\activate
```

5. Instalar todas las dependencias:

```
pip install -r requirements.txt
```

6. Ejecutar la aplicación mediante:

```
python main.py
```

Tras completar estos pasos aparecerá la ventana principal de la aplicación, desde la cual podrá comenzarse el análisis de vídeos.

E.4. Manual del usuario

La aplicación presenta una interfaz sencilla organizada en diferentes áreas funcionales.

Al iniciar el programa se muestra la ventana principal desde la cual el usuario podrá realizar todas las operaciones disponibles.

Carga de un vídeo

Para comenzar un análisis es necesario pulsar el botón **Cargar vídeo**.

Se abrirá el explorador de archivos del sistema operativo, permitiendo seleccionar un vídeo compatible con la aplicación.

Los formatos soportados son:

- MP4
- AVI
- MOV
- MKV

Una vez cargado el vídeo, se visualizará el primer fotograma y se habilitarán las opciones de procesamiento.

Selección del modelo

Antes de iniciar el procesamiento, el usuario podrá seleccionar el modelo de estimación de pose que desea utilizar.

La arquitectura de la aplicación permite incorporar diferentes modelos, facilitando la comparación entre ellos.

Procesamiento del vídeo

Tras pulsar el botón **Procesar vídeo**, la aplicación analizará automáticamente todos los fotogramas.

Durante el procesamiento se mostrarán:

- El vídeo original.

- El esqueleto detectado sobre cada fotograma.
- El progreso del procesamiento.
- Las métricas calculadas en tiempo real.
- Las anomalías detectadas.

Visualización de resultados

Una vez finalizado el análisis, la aplicación mostrará diferentes indicadores estadísticos entre los que destacan:

- Número total de fotogramas procesados.
- Número de fotogramas válidos.
- Confianza media obtenida.
- Velocidad de procesamiento.
- Número de anomalías detectadas.

Estos datos permiten obtener una visión general del rendimiento del modelo utilizado.

Exportación de resultados

La aplicación permite guardar los resultados obtenidos para su posterior análisis.

Actualmente pueden exportarse:

- Coordenadas de los keypoints en formato CSV.
- Registro de anomalías detectadas.

Los archivos generados se almacenarán en la ubicación seleccionada por el usuario.

Reinicio del procesamiento

En cualquier momento es posible reiniciar el análisis utilizando el botón **Reiniciar**.

Esta opción elimina los resultados obtenidos hasta el momento y permite comenzar un nuevo procesamiento utilizando el mismo vídeo o cargando uno diferente.

Finalización de la aplicación

Para cerrar el programa únicamente será necesario cerrar la ventana principal.

La aplicación liberará automáticamente todos los recursos utilizados durante la ejecución, incluyendo el vídeo cargado y los modelos de estimación de pose que se encuentren inicializados.

Apéndice F

Anexo de sostenibilización curricular

F.1. Introducción

Durante la realización de este Trabajo Fin de Grado he podido comprobar que la sostenibilidad no se limita únicamente al ámbito medioambiental, sino que también está relacionada con el impacto que la tecnología puede tener sobre la sociedad y la calidad de vida de las personas.

El objetivo principal del proyecto ha sido desarrollar una aplicación capaz de analizar el movimiento humano mediante técnicas de visión por computador y estimación de pose, con una posible aplicación en el estudio de enfermedades neurodegenerativas como la Esclerosis Lateral Amiotrófica (ELA). Aunque se trata de un proyecto tecnológico, considero que puede aportar herramientas útiles para facilitar el trabajo de investigadores y profesionales sanitarios, permitiendo obtener información objetiva sobre el movimiento de una persona de una forma sencilla y poco invasiva.

Uno de los aspectos que más me ha hecho reflexionar durante el desarrollo del proyecto ha sido el potencial que tiene la inteligencia artificial cuando se utiliza para mejorar la salud y el bienestar de las personas. Este tipo de tecnologías no deben entenderse como un sustituto del trabajo realizado por los profesionales sanitarios, sino como una herramienta de apoyo que puede ayudar a obtener información adicional y facilitar determinadas tareas de análisis.

Otro aspecto importante ha sido el uso de herramientas de código abierto como Python, OpenCV o PySide6. Gracias a ellas ha sido posible desarrollar

toda la aplicación sin depender de software propietario, favoreciendo que cualquier estudiante, investigador o desarrollador pueda reutilizar el trabajo realizado, aprender a partir de él o continuar mejorándolo en el futuro.

También he intentado que la aplicación estuviera diseñada de forma modular, permitiendo incorporar nuevos modelos de estimación de pose sin necesidad de modificar completamente el proyecto. Creo que desarrollar software pensando en su reutilización y mantenimiento también forma parte de una forma de trabajar más sostenible, ya que evita duplicar esfuerzos y facilita que otros puedan aprovechar el trabajo ya realizado.

Durante el desarrollo también he sido consciente de que los modelos de inteligencia artificial requieren una importante capacidad de procesamiento. Por ello, siempre que ha sido posible se han utilizado modelos previamente entrenados, evitando realizar procesos de entrenamiento que implican un elevado consumo de recursos computacionales. Además, el procesamiento de los vídeos se realiza de manera secuencial, analizando únicamente un fotograma cada vez, lo que permite reducir el consumo de memoria y mejorar la eficiencia de la aplicación.

Por último, este proyecto me ha permitido comprender la importancia de desarrollar tecnología de forma responsable. Cuando se trabaja con aplicaciones relacionadas con la salud es fundamental recordar que los resultados obtenidos deben interpretarse siempre con cautela y nunca sustituir el criterio de un profesional. Del mismo modo, cualquier utilización con pacientes reales debe garantizar la protección de sus datos personales y respetar siempre la normativa vigente.

En definitiva, considero que este Trabajo Fin de Grado no solo me ha permitido aplicar conocimientos técnicos adquiridos durante la carrera, sino también reflexionar sobre cómo la ingeniería informática puede contribuir al bienestar de las personas cuando se desarrolla con responsabilidad, pensando en su utilidad social, en la reutilización del software y en el uso eficiente de los recursos disponibles.

Bibliografía

- [1] NumPy Developers. Numpy reference guide. <https://numpy.org/doc/>, 2025. Último acceso: junio de 2025.
- [2] Pandas Development Team. Pandas documentation. <https://pandas.pydata.org/docs/>, 2025. Último acceso: junio de 2025.
- [3] TensorFlow. Tensorflow lite documentation. <https://www.tensorflow.org/lite>, 2025. Último acceso: junio de 2025.
- [4] Ultralytics. Ultralytics yolo documentation. <https://docs.ultralytics.com/>, 2025. Último acceso: junio de 2025.